Technical University of Denmark

DTU Compute
Department of Applied Mathematics and Computer Science

Examination Preparation

Test Your Database Skills and Prepare for the Exam!

©Flemming Schmidt

These slides have been updated by Anne Haxthausen
2018+2019

1. The Relational Model

1.1 Attributes can be simple, composite, multivalued or derived. Please explain and exemplify.

1.2 Explain and exemplify the concept of a Super Key, Candidate Key and Primary Key.

1.3 Explain and exemplify the concept of a Foreign Key.

1.4 Explain and exemplify the concept of Relation Schema and Relation Instance.

1.5 Draw the Database Schema Diagram from the Relation Schemas:
Instructor(InstID, InstName, Birth)
Student(StudID, StudName, Birth)
Advisor(StudID, InstID)

1.6 Consider the tabel Parts:

PartNo	Qty
1001	9
1002	3
1003	Null
1004	4
1005	Null
1006	8

List the results from the SQL Queries:
a) SELECT SUM(Qty), AVG(Qty), COUNT(Qty) FROM Parts;
b) SELECT COUNTS(*) FROM Parts;

Q&A: 1. The Relational Model

1.1 Attributes can be simple, composite, multivalued or derived. Please explain and exemplify.

1.1 A **simple attribute** has atomic attribute values selected from a simple domain (e.g. PartNumber or Quantity).
A **composite attribute** can be represented by its component attributes (e.g. Name can be represented by the attributes: FirstName, MiddleName and LastName).
A **multivalued attribute** has a set of values (e.g. a Person has multiple PhoneNumbers).
A **derived attribute** can be calculated from other attributes (e.g. TotalPrice is Quantity multiplied with UnitPrice).

Q&A: 1. The Relational Model

1.2 Explain and exemplify the concept of a Super Key, Candidate Key and Primary Key.

1.2 A **Super Key** is a set of attributes, whose values uniquely determines each row in a table (i.e. tuple in a relation).
A **Candidate Key** is a minimal Super Key; That is, a minimum set of attributes, whose values uniquely determines each row in a table.
A **Primary Key** is a Candidate Key chosen by the database designer, to uniquely determine each row in a table.

1.3 Explain and exemplify the concept of a Foreign Key.

1.3 A **Foreign Key** is an attribute where attribute values must appear in another attribute usually in another table. For instance, in the Relation Schemas:
Instructor(InstID, InstName, DeptName, Salary)
Department(DeptName, Building, Budget)
Values in Instructor.DeptName must appear in the attribute Department.DeptName.
Instructor is the referencing table to the referenced table Department.

Q&A: 1. The Relational Model

1.4 Explain and exemplify the concept of Relation Schema and Relation Instance.

1.5 Draw the Database Schema Diagram from the Relation Schemas:
Instructor(InstID, InstName, Birth)
Student(StudID, StudName, Birth)
Advisor(StudID, InstID)

1.6 Consider the tabel Parts:

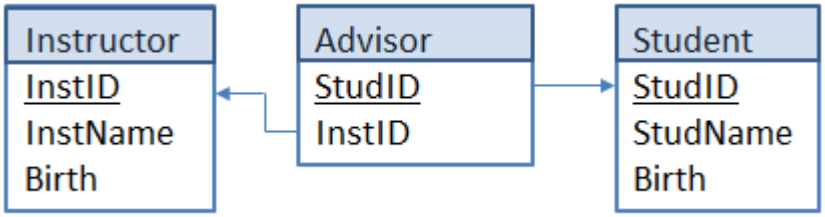
PartNo	Qty
1001	9
1002	3
1003	Null
1004	4
1005	Null
1006	8

List the results from the SQL Queries:
a) SELECT SUM(Qty), AVG(Qty), COUNT(Qty) FROM Parts;
d) SELECT COUNTS(*) FROM Parts;

1.4 The definition of a table, with its attributes and Primary Key is called a Relation Schema, and the actual content of a table at a specific point in time is called a Relation Instance. For instance:
Relation Schema: Advisor(StudID, InstID)
Relation Instance:

StudID	InstID
12345	10101
44553	22222

1.5 A Database Schema Diagram consists of names of Relations, Attributes and Primary Keys and arrows for Foreign Keys:



1.6 Results from the queries:
a) 24, 6, 4 Note: AVG is SUM/4 not SUM/6!
d) 6

2. Introductory SQL

A Database named Employees has the following Relation Schemas:

Employee(EmpName, EmpAddress, Birth)
Company(ComName, ComAddress)
Works(EmpName, ComName, Salary)

2.1 Write the SQL statements that was used to create the database and start using it.

2.2 In SQL create the relation Works. Identify any referential-integrity constraints that should hold. If the referenced relation row is deleted the referencing relation row should also be deleted.

2.3 In SQL insert data into the relation Employee about Hans Andersen living at Nystedvej 18, 2500 Valby and having an unknown date of birth.

2.4 In SQL create an additional attribute named Education to the table Employee.

2.5 In SQL update the date of birth for Hans Andersen to 8. April 1984 and his education to Certified Painter.

2.6 In SQL for each employee return the name, address and age.

2.7 In SQL return the name and education of all employees that have a certified education.

2.8 In SQL return the average employee salary for each company.

2.9 In SQL return a table with the attributes EmpName, EmpAddress, ComName, ComAddress, Salary, Birth and Education.

Q&A: 2. Introductory SQL

A Database named Employees has the following Relation Schemas:
Employee(EmpName, EmpAddress, Birth)
Company(ComName, ComAddress)
Works(EmpName, ComName, Salary)

2.1 Write the SQL statements that was used to create the database and start using it.

2.2 In SQL create the relation Works. Identify any referential-integrity constraints that should hold. If the referenced relation row is deleted the referencing relation row should also be deleted.

2.3 In SQL insert data into the relation Employee about Hans Andersen living at Nystedvej 18, 2500 Valby and having an unknown date of birth.

2.1 CREATE DATABASE Employees;
USE Employees;

2.2 CREATE TABLE Works(
EmpName VARCHAR(50),
ComName VARCHAR(50),
Salary NUMERIC(10,2),
PRIMARY KEY (EmpName, ComName),
FOREIGN KEY (EmpName) REFERENCES
Employee (EmpName) ON DELETE CASCADE,
FOREIGN KEY (ComName) REFERENCES
Company (ComName) ON DELETE CASCADE);

2.3 INSERT INTO Employee VALUES ('Hans Andersen', 'Nystedvej 18, 2500 Valby', Null);

Q&A: 2. Introductory SQL

2.4 In SQL create an additional attribute named Education to the table Employee.

2.4 ALTER TABLE Employee ADD Education VARCHAR(50);

2.5 In SQL update the date of birth for Hans Andersen to 8. April 1984 and his education to Certified Painter.

2.5 UPDATE Employee SET Birth = '19840408', Education = 'Certified Painter'
WHERE EmpName = 'Hans Andersen';

2.6 In SQL for each employee return the name, address and age.

2.6 SELECT EmpName, EmpAddress, TIMESTAMPDIFF(YEAR, Birth, CURDATE()) AS AGE FROM Employee;

2.7 In SQL return the name and education of all employees that have a certified education.

2.7 SELECT EmpName, Education FROM Employee
WHERE Education LIKE '%Certified%';

2.8 In SQL return the average employee salary for each company.

2.8 SELECT ComName, AVG(Salary) FROM Works GROUP BY ComName;

2.9 In SQL return a table with the attributes EmpName, EmpAddress, ComName, ComAddress, Salary, Birth and Education.

2.9 SELECT EmpName, EmpAddress, ComName, ComAddress, Salary, Birth, Education FROM (Works LEFT OUTER JOIN Company USING (ComName)) LEFT OUTER JOIN Employee USING (EmpName);

3. Intermediate SQL

3.1 Display a list of all students, showing their ID, name and the number of courses that they have taken using a JOIN operation .

Student(StudID, StudName, Birth, DeptName, TotCredits)

Takes(StudID, CourseID, SectionID, Semester, StudyYear, Grade)

3.2 Write the same query as above, but use a scalar subquery (i.e. a SELECT statement that returns a single value for a number of rows) within another SELECT statement.

3.3 Create a View of Students :

YoungStudents(StudID, StudName, Birth)

of students with birth after 1993 and do not include students Levy and Sanches.

3.4 Create the database users Anders and Bent and give them the initial, temporary password 'PleaseChange'.

Grant Anders read access and Bent all privileges to the University DB.

Logon as Anders and change password to a password of your liking.

Q&A: 3. Intermediate SQL

3.1 Display a list of all students, showing their ID, name and the number of courses that they have taken using a JOIN operation .

Student(StudID, StudName, Birth, DeptName, TotCredits)

Takes(StudID, CourseID, SectionID, Semester, StudyYear, Grade)

3.2 Write the same query as above, but use a scalar subquery (i.e. a SELECT statement that returns a single value for a number of rows) within another SELECT statement.

3.3 Create a View of Students :

YoungStudents(StudID, StudName, Birth)

of students with birth after 1993 and do not include students Levy and Sanches.

3.1 SELECT StudID, StudName, COUNT(CourseID) AS CoursesTaken FROM Student NATURAL LEFT OUTER JOIN Takes GROUP BY StudID;

3.2 SELECT StudID, StudName, (SELECT COUNT(*) FROM Takes WHERE Takes.StudID = Student.StudID) AS CourseTaken FROM Student;

3.3 CREATE VIEW YoungStudents AS SELECT StudID, StudName, Birth FROM Student WHERE Birth > 19931231 AND StudName NOT IN ('Levy', 'Sanches');

Q&A: 3. Intermediate SQL

3.4 Create the database users Anders and Bent and give them the initial, temporary password ‘PleaseChange’.

Grant Anders read access and Bent all privileges to the University DB.

Logon as Anders and change password to a password of your liking.

3.4 Login as root (i.e. DBA) and type:

```
CREATE USER 'Anders'@'localhost'  IDENTIFIED BY
'PleaseChange';
CREATE USER 'Bent'@'localhost' IDENTIFIED BY
'PleaseChange';
GRANT SELECT ON University.* TO
'Anders'@'localhost';
GRANT ALL ON University.* TO 'Bent'@'localhost';
```

Login by Anders:

```
SET PASSWORD = Password('Sec4525‘);
```

Note, above you need to use 'Sec4525' instead of Password('Sec4525') for some MySQL versions (starting from version 5.7.6).

4. Advanced SQL

4.1 Define a **Trigger** Student_After_Insert that after a student has been inserted into the table Student will insert the row in the table StudentLog with a timestamp added.
The Relation Schemas:
Student(StudID, StudName, Birth, DeptName, TotCredits)

StudentLog(StudID, StudName, Birth, DeptName, TotCredits, LogTime)

4.2 Define a **Function** InstructorSalary that takes an instructor ID as argument and returns the salary of the instructor.
Relation Schema:
Instructor(InstID, InstName, DeptName, Salary)

In all the answers to 4.1 to 4.5 expect the trick:
DELIMITER //
SQL statements //
DELIMITER ;
To allow multiple input statements to end with ';'.

4.3 Define a simple **Procedure** called StudentBackup that will copy all rows from the table Student to the table StudentOld, and then delete all rows in the table StudentLog.
Relation Schemas:
Student(StudID, StudName, DeptName, Birth, TotCredits)
StudentOld(StudID, StudName, DeptName, Birth, TotCredits)
StudentLog(StudID, StudName, DeptName, Birth, TotCredits, LogTime)

4.4 Redefine the procedure StudentBackup and name it StudBackup to include **error handling** and a **transaction**.

4.5 Program an **event** called BackupMonthly that automatically will execute the procedure StudentBackup after each month end, starting 1. June 2018.

Q&A: 4. Advanced SQL

4.1 Define a **Trigger** Student_After_Insert that after a student has been inserted into the table Student will insert the row in the table StudentLog with a timestamp added.
The Relation Schemas:
Student(StudID, StudName, Birth, DeptName, TotCredits)

StudentLog(StudID, StudName, Birth, DeptName, TotCredits, LogTime)

4.2 Define a **Function** InstructorSalary that takes an instructor ID as argument and returns the salary of the instructor.
Relation Schema:
Instructor(InstID, InstName, DeptName, Salary)

4.1 CREATE TRIGGER Student_After_Insert
AFTER INSERT ON Student FOR EACH ROW
BEGIN
INSERT StudentLog VALUES (NEW.StudID, NEW.StudName, NEW.Birth, NEW.DeptName, NEW.TotCredits, NOW());
END;

4.2 CREATE FUNCTION InstructorSalary (
vInstID VARCHAR(5))
RETURNS INTEGER
BEGIN
DECLARE vSalary INTEGER;
SELECT Salary INTO vSalary FROM Instructor
WHERE Instructor.InstID = vInstID;
RETURN vSalary;
END;

Q&A: 4. Advanced SQL

4.3 Define a simple **Procedure** called StudentBackup that will copy all rows from the table Student to the table StudentOld, and then delete all rows in the table StudentLog.
Relation Schemas:
Student(StudID, StudName, DeptName, Birth, TotCredits)
StudentOld(StudID, StudName, DeptName, Birth, TotCredits)
StudentLog(StudID, StudName, DeptName, Birth, TotCredits, LogTime)

4.4 Redefine the procedure StudentBackup and name it StudBackup to include **error handling** and a **transaction**.

```
4.3 CREATE PROCEDURE StudentBackup ()
BEGIN
DELETE FROM StudentOld;
INSERT INTO StudentOld SELECT * FROM Student;
DELETE FROM StudentLog;
END;
```

```
4.4 CREATE PROCEDURE StudBackup ()
BEGIN
DECLARE vSQLSTATE CHAR(5) DEFAULT '00000';
DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
  BEGIN
     GET DIAGNOSTICS CONDITION 1
     vSQLSTATE = RETURNED_SQLSTATE;
  END;
START TRANSACTION;
DELETE FROM StudentOld;
INSERT INTO StudentOld SELECT * FROM Student;
DELETE FROM StudentLog;
SELECT vSQLSTATE;
IF vSQLSTATE = '00000' THEN COMMIT; ELSE
ROLLBACK;
END IF;
END;
```

Q&A: 4. Advanced SQL

4.5 Program an **event** called BackupMonthly that automatically will execute the procedure StudentBackup after each month end, starting 1. June 2018.

4.5 CREATE EVENT BackupMonthly
ON SCHEDULE EVERY 1 MONTH
STARTS '2018-06-01 00:00:01'
DO CALL StudentBackup;

SET GLOBAL event_scheduler = 1;

5. Entity-Relationship Diagrams

The owner of the Small Shop stated that he has clients registered by client number, name and address, to which he sells various items registered by item number, name and unitprice. He tracks all deliveries by client, item, quantity, subtotals of items and delivery date.

5.1 Draw an Entity-Relationship Diagram using the Textbook Adapted UML Notation.

5.2 Describe the order, cardinality and totality (i.e. Entity Participation) of the relationship Deliveries.

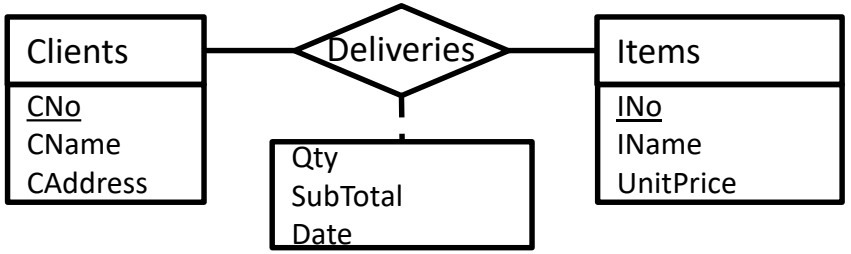
5.3 Construct the Relation Schemas

5.4 Construct a Database Instance

Q&A 5. Entity-Relationship Diagrams

5.1 Draw an Entity-Relationship Diagram using the Textbook Adapted UML Notation.

5.1 Entity-Relationship Diagram



5.2 Describe the order, cardinality and totality (i.e. Entity Participation) of the relationship set Deliveries.

5.2 Order: There is a *Binary Relationship* between the Entity Sets Clients and Items.
Cardinality: *Many-to-Many*. A Client can buy many Items, and an Item can be bought by many Clients.
Totality: Entities Clients and Items have *partial* participation in the relationship, since a Client might not have bought anything yet, and an Item might not have been bought by anyone.

5.3 Construct the Relation Schemas

5.3 Relation Schemas
Clients(CNo, CName, CAddress)
Items(INo, IName, UnitPrice)
Deliveries(CNo, INo, Qty, Date)

Note SubTotal is not included in Deliveries as it is a derived attribute. Date is included in the primary key to allow the a client to get the same delivery several times.

Q&A 5. Entity-Relationship Diagrams

5.4 Construct a Database Instance

5.4 Relation Instance

Clients

CNo	CName	CAddress
1050	Adam Asimov	Høgholtvej 5, 2500 Valby
1051	Brian Balter	Hjortevej 17, 4690 Haslev
1052	Casper Cortz	Azaleavej 24, 2000 Kbh F

Items

INo	IName	UnitPrice
431001	Dishwasher	8.399,00
431002	Cooler	3.299,00
451001	Mounting	47,50

Deliveries

	CNo	INo	Qty	Date
73201	1050	431002	4	2017-02-17
73202	1050	451001	2	2017-02-17
73203	1052	431002	3	2017-03-15

6. Normalization

6.1 Normalize the relation schema to 4NF:
Members(MemberID, MemberName, PhoneNo, GroupID, GroupName, Birth)

The associated constraints:
MemberID → MemberName, GroupID, Birth
MemberID →→ PhoneNo
GroupID → GroupName

6.2 Use Armstrong's rules to make a list of Candidate Keys for the Relation Schema:
R(A, B, C, D)
with the Functional Dependencies:
A → CD, D → B, B → A.
Please specify which of Armstrong's rules are used in each step. Finally select a Primary Key.

Q&A 6. Normalization

6.1 Normalize the relation schema to 4NF:
Members(MemberID, MemberName, PhoneNo, GroupID, GroupName, Birth)

The associated constraints:
MemberID → MemberName, GroupID, Birth
MemberID →→ PhoneNo
GroupID → GroupName

6.1 Relation Schema Normalized

All attributes except PhoneNo functionally dependent on MemberID, so the combination of MemberID and PhoneNo must be chosen as the primary key for Members:

Members(MemberID, MemberName, PhoneNo, GroupID, GroupName, Birth)

This relation is not in 2NF as the non-primary attributes MemberName, GroupID, GroupName, and Birth only depend partially on the key (only on MemberID, not on PhoneNo). Normalizing to 2NF gives:

Phones(MemberID, PhoneNo)

Members(MemberID, MemberName, GroupID, GroupName, Birth)

Phones is in 3NF, but Members is not 3NF as there is a transitive dependency:

MemberID → GroupID → GroupName

Normalization to 3NF gives:

Phones(MemberID, PhoneNo)

Members(MemberID, MemberName, GroupID, Birth)
Groups(GroupID, GroupName)

The two last relations are in 4 NF as they do not contain any multivalued dependencies. Phones is also in 4NF as MemberID →→ PhoneNo is trivial. (a →→ b is *trivial* for relation R, if b is a subset of a, or a union b consists of all attributes of R.)

Q&A 6. Normalization

6.2 Use Armstrong's rules to make a list of Candidate Keys for the Relation Schema:
R(A, B, C, D)
with the Functional Dependencies:
A → CD, D → B, B → A.
Please specify which of Armstrong's rules are used in each step. Finally select a Primary Key.

6.2 Amstrong's rules

1. A → A by rule self-determination
2. A → C by rule decomposition of A → CD
3. A → D by rule decomposition of A → CD
4. A → B given A → D and given D → B then by the transitivity rule
5. A → ABCD by rule union of the above
6. B → ABCD given B → A and A → ABCD by the transitivity rule
7. D→ ABCD given D → B and B → ABCD by the transitivity rule.

Candidate keys are A, B and D, and one of them can be selected as Primary Key. I select attribute A to be the Primary Key.

7. Formal Query Languages

7.1 A relation has the relation schema:
Student(StudID, StudName, DeptName, TotCredits)

Consider the query:
"Find the ID and name for students, whose total credits is greater than 60".
State the query in SQL, Relational Algebra, Tuple Calculus and Domain Calculus.

7.2 List the Basic Operations of Relational Algebra.

7.3 Prove how Set Intersection can be derived from the Basic Operations.

7.4 If R represents a finite relation and A represents an attribute in R, the please state if the Tuple Calculus expression:

$$\{ t \mid t \in R \wedge t[A] \neq 5 \}$$

is a safe expression and why/why not.

Q&A: 7. Formal Query Languages

7.1 A relation has the relation schema:
Student(StudID, StudName, DeptName, TotCredits)

Consider the query:
"Find the ID and name for students, whose total credits is greater than 60".
State the query in SQL, Relational Algebra, Tuple Calculus and Domain Calculus.

7.1 **SQL**

```
SELECT StudID, StudName FROM Student
WHERE TotCredits > 60;
```

Relational Algebra

$\prod_{\text{StudID, StudName}}(\sigma_{\text{TotCredits} > 60}(\text{Student}))$

Tuple Calculus

$\{t \mid \exists s \in \text{Student} ($
$t[\text{StudID}] = s[\text{StudID}]$
$\wedge\ t[\text{StudName}] = s[\text{StudName}]$
$\wedge\ s[\text{TotCredits}] > 60)\}$

Domain Calculus

$\{ < id, na > \mid \exists dn, tc ($
$< id, na, dn, tc > \in \text{student} \wedge tc > 60)\}$

Q&A: 7. Formal Query Languages

7.2 List the Basic Operations of Relational Algebra.

7.2 The Basic Operations of Relational Algebra are Selection, Projection, Union, Set Difference, Cartesian Product and Rename.

7.3 Prove how Set Intersection can be derived from the Basic Operations.

7.3 Set Intersection can be derived from Set Difference alone: $r \cap s = r - (r - s)$!
In the parentheses the tuples found in r but not in s are returned by Set Difference (-). The Set Difference between r and these tuples only found in r, then will result in the tuples in r, which are also found in s. The result is the tuples that are found both in r and in s, the Set Intersection.

7.4 If R represents a finite relation and A represents an attribute in R, the please state if the Tuple Calculus expression:

$$\{ t \mid t \in R \wedge t[A] \neq 5 \}$$

is a safe expression and why/why not.

7.4 The expression is a safe expression. For the total predicate to return the truth value **True**, both sub predicates as arguments to the connective **AND** need to be true. Since R has a limited number of tuples, the total predicate result can at a maximum have that number. Thus a limited number of tuples will qualify, and the expression is a safe expression.

8. Indexing and Hashing

See [demo] exercises in slide set 11.

- Best wishes for a successful examination

Anne Haxthausen